

Performance & Code Optimization Notes

FA0 IDEA Frontend Developer Technical Test – Andrea Giovanni Perozziello

1. Summary of the Original Code

The initial component was functional but had several issues:

- API logic was embedded directly in the component
 - Multiple `setLoading(false)` calls could cancel each other out
 - Duplicate `fetch` blocks meant repeated logic and more maintenance overhead
 - No caching or performance optimization was in place
 - Error handling was missing
-

2. Key Improvements Made

Code Refactoring

- **Encapsulated API logic** into a reusable service (`api.js`)
- Removed `useEffect` and `useState` in favor of `useQuery` (React Query v5)

State & Data Management

- Integrated **React Query** for:
 - Caching API results
 - Managing loading/error states
 - Handling retries automatically

Performance Optimization

- Used `useMemo` to prevent re-renders when user/project data didn't change
- Ensured the component re-renders only on actual data change

Error Handling

- Added basic try/catch logic and fallback UI messaging
-

3. Performance Testing Approach

I compared the original and refactored behavior using:

- **Browser DevTools:**
 - Watched for repeated network calls on re-renders
 - Verified that React Query prevented unnecessary fetches
 - **React DevTools:**
 - Checked component re-renders and ensured minimal updates
 - **Simulated API Delays:**
 - Added random errors and latency to test loading/error states
-

4. Why This Version is Better

Feature	Before	After
API structure	Repeated fetch logic	Centralized in api.js
State handling	Manual w/ useState	Automatic via React Query
Caching	None	Query-based caching
Re-renders	Frequent	Memoized lists
Error handling	Missing	Present
Maintainability	Low	Modular and scalable

5. Tools & Libraries Used

- **React Query v5** – for caching, async state, and performance
- **React hooks (useMemo)** – to reduce unnecessary rendering
- **Mock API** – using local data + `setTimeout` to simulate real latency

6. Future Optimizations

- Implement pagination or infinite scroll for large datasets
- Add global error boundary
- Add loading skeletons instead of basic `<p>Loading...</p>`
- Write unit tests for the API layer and UI rendering logic

Thanks for reading, and I hope this shows the thinking behind my decisions. I tried to balance simplicity with best practices and clarity.